

Ejercitación 1:

1. ¿Qué es CSS y para qué se usa?

CSS son las siglas de Hojas de Estilo en Cascada. No es un lenguaje de programación ni un lenguaje de marcado, sino una tecnología y un "lenguaje de hojas de estilo". Es un estándar aceptado por la industria web (W3C). Su objetivo principal es separar la información (el contenido escrito en HTML) de su forma de representarse (el diseño o formato visual). Se usa para controlar la apariencia de la página web y describir cómo se presentan los elementos al usuario. Con CSS se pueden especificar estilos precisos como el tamaño, las fuentes, el color, el espaciado entre textos y recuadros, y la ubicación exacta donde disponer texto e imágenes en la página.

2. CSS utiliza reglas para las declaraciones de estilo, ¿cómo funcionan?

El modo de funcionamiento de CSS se basa en una sintaxis especial que le dice al navegador cómo mostrar los elementos. Estas reglas funcionan mediante tres componentes básicos:

- **El selector:** Le indica al navegador qué parte del documento HTML se verá afectada por esa regla (por ejemplo, escribiendo h1 para afectar a los títulos principales). Los selectores se pueden escribir individualmente o agruparse separándolos con comas para aplicar el mismo estilo a varios a la vez (por ejemplo: h1, h2, h3 { color: red; }).
- **La propiedad:** Especifica qué aspecto visual del elemento se va a cambiar (por ejemplo, el color o la alineación con text-align). Todas las propiedades que se deseen modificar para un selector se agrupan entre llaves { } y, si hay varias, es obligatorio separarlas entre sí con un punto y coma (;).
- **El valor:** Es lo que establece la modificación concreta de la propiedad (por ejemplo, el valor red para la propiedad color). Una regla de sintaxis es que, si el valor está formado por más de una palabra, hay que ponerlo siempre entre comillas (por ejemplo: font-family: "sans serif";).

3. ¿Cuáles son las tres formas de dar estilo a un documento?

A nivel de elemento HTML: Consiste en definir los estilos directamente dentro de la etiqueta HTML utilizando la propiedad style. Es la forma más fácil, pero la menos recomendada, y se suele usar para hacer pruebas rápidas de diseño antes de trasladar el código a la zona definitiva.

A nivel de página (En el <head>): Consiste en agrupar la definición de los estilos para los distintos elementos de la página dentro de una sección especial en la cabecera del documento. Estas reglas se encierran obligatoriamente entre las etiquetas <style> y </style>.

En un archivo externo independiente: Es la metodología más empleada y recomendada. Consiste en agrupar las reglas de estilo en un archivo separado que debe tener la extensión .css. Luego, este archivo se vincula al documento HTML mediante la etiqueta <link rel="StyleSheet" href="..." type="text/css">.

4. ¿Cuáles son los distintos tipos de selectores más utilizados? Ejemplifique cada uno.

Los más utilizados y fundamentales son:

- **Selector universal:** Se representa con un asterisco (*) y sirve para aplicar un estilo a absolutamente cualquier elemento de la estructura del documento. Ejemplo: * { color: red; } (Aplica el color rojo a todos los elementos).
 - **Selectores de tipos:** Se corresponden directamente con el nombre de una etiqueta HTML (o tipo de elemento). Este selector equivale a cada instancia de ese elemento en todo el documento. Ejemplo: H1 { font-family: sans-serif } (Aplica esa fuente a todos los títulos H1).
 - **Selectores de clases:** Permiten aplicar estilos utilizando el atributo "class" de HTML. Se utiliza un punto (.) seguido del nombre de la clase deseada. Ejemplo: .pastoral { color: green } (Aplica el color verde a todos los elementos que tengan el atributo class="pastoral"). También se puede combinar con un tipo: H1.pastoral { color: green } (Solo aplica a los H1 que tengan esa clase).
 - **Selectores de ID:** Se utilizan para aplicar estilos a una instancia única de un elemento basado en su identificador exclusivo (el atributo "id"). Se escribe un numeral (#) seguido del valor del ID. Ejemplo: #capitulo1 { text-align: center } o H1#capitulo1 { text-align: center } (Aplica la alineación centrada solo al elemento que tiene el id="capitulo1").
5. **¿Qué es una pseudo-clase? Cuáles son las más utilizadas aplicadas a vínculos?**

Las pseudo-clases sirven para clasificar o aplicar estilos a los elementos basándose en características dinámicas o información que no se puede deducir simplemente mirando la estructura del código HTML (más allá de su nombre, atributos o contenido). Por ejemplo, un elemento puede adquirir o perder una pseudo-clase a medida que el usuario interactúa con la página. Al igual que los pseudo-elementos, no aparecen escritas en el código fuente HTML original.

Las aplicaciones (navegadores) suelen mostrar los hipervínculos de forma distinta según si se interactuó con ellos o no. Para controlarlo, CSS proporciona las siguientes pseudo-clases específicas para enlaces (normalmente la etiqueta A en HTML):

- **:link:** Se aplica a los vínculos que aún no han sido visitados por el usuario.
- **:visited:** Se aplica una vez que el vínculo ya ha sido visitado por el usuario. (Los estados :link y :visited son mutuamente excluyentes).
- **:hover:** Es una pseudo-clase dinámica que se aplica mientras el usuario "señala" el elemento (por ejemplo, cuando pasa el puntero por encima del vínculo), pero sin llegar a hacer clic.
- **:active:** Se aplica en el momento exacto en que el elemento está siendo activado por el usuario (por ejemplo, el breve lapso de tiempo en el que se presiona y se suelta el botón del mouse sobre el vínculo).

6. **¿Qué es la herencia en CSS?**

La herencia es un mecanismo mediante el cual algunos valores de estilo son "heredados" por los elementos "hijos" a partir de sus elementos "padres" en la estructura del documento. Cada propiedad de CSS define específicamente si es heredable o no. Por ejemplo, los valores especificados mediante porcentajes no se heredan directamente, pero sus valores computados finales sí.

CSS permite forzar esta herencia escribiendo el valor inherit, lo que significa que el elemento tomará obligatoriamente el mismo valor que tiene la propiedad en su elemento padre.

7. ¿En qué consiste el proceso denominado cascada?

La cascada es el proceso mediante el cual CSS asigna una "fuerza" a cada regla de estilo para determinar cuál tiene prioridad cuando varias de ellas se aplican al mismo tiempo sobre un mismo elemento. Los estilos pueden provenir de tres orígenes distintos que se superponen: el autor (quien crea la página), el usuario (quien la lee) y la aplicación del usuario (el navegador predeterminado). Cuando hay un conflicto entre reglas, la cascada aplica un orden para decidir qué valor se muestra finalmente:

1. **Fuerza y origen:** Por defecto, las reglas definidas por el autor prevalecen sobre las del usuario, y estas a su vez sustituyen a las predeterminadas del navegador. Sin embargo, al emplear la declaración !important, esta jerarquía se invierte para otorgarle prioridad a las configuraciones del usuario.
2. **Especificidad del selector:** Los selectores con mayor nivel de especificidad (como un identificador o ID) tienen precedencia sobre aquellos más generales (como una etiqueta HTML estándar).
3. **Orden de declaración:** En el caso de que dos o más reglas posean exactamente la misma fuerza, origen y especificidad, se aplicará aquella que haya sido definida en último lugar dentro del código.

Ejercitación 2:

Analizar el siguiente código señalando declaraciones y aplicaciones de reglas, y su efecto.

```
p#normal {  
    font-family: arial, helvetica;  
    font-size: 11px;  
    font-weight: bold;  
}  
*#destacado {  
    border-style: solid;  
    border-color: blue;  
    border-width: 2px;  
}  
#distinto {  
    background-color: #9EC7EB;  
    color: red;  
}  
<p id="normal">Este es un párrafo</p>  
<p id="destacado">Este es otro párrafo</p>  
<table id="destacado"><tr><td>Esta es una tabla</td></tr></table>  
<p id="distinto">Este es el último párrafo</p>
```

Análisis de las reglas CSS

p#normal { ... }

- **Selector:** Combina un selector de tipo (p) con un selector de ID (#normal).
- **Declaraciones:** Familia tipográfica Arial o Helvetica, tamaño 11px y grosor negrita.
- **Efecto:** Solo se aplica a los párrafos <p> que tengan exactamente id="normal".

***#destacado { ... }**

- **Selector:** Combina el selector universal (*) con el selector de ID #destacado. El * puede omitirse cuando va junto a un ID, ya que su significado es equivalente.
- **Declaraciones:** Borde de estilo sólido, color azul y grosor de 2px.
- **Efecto:** Aplica ese borde a cualquier elemento HTML que tenga id="destacado", sin importar su tipo de etiqueta.

#distinto { ... }

- **Selector:** Selector de ID puro, equivalente a *#distinto.
- **Declaraciones:** Color de fondo #9EC7EB (celeste) y color de texto rojo.
- **Efecto:** Al elemento con id="distinto" se le aplica fondo celeste y texto en color rojo.

Análisis HTML

- **<p id="normal">Este es un párrafo</p>:** Hace referencia a p#normal. El texto se verá en Arial/Helvetica, 11px y en negrita.
- **<p id="destacado">Este es otro párrafo</p>:** Hace referencia a *#destacado. El párrafo tendrá un borde sólido azul de 2px.
- **<table id="destacado"><tr><td>Esta es una tabla</td></tr></table>:** También hace referencia a *#destacado, ya que el selector universal abarca la etiqueta <table>. La tabla tendrá el mismo borde azul de 2px.
- **<p id="distinto">Este es el último párrafo</p>:** Hace referencia a #distinto. El párrafo tendrá fondo celeste y texto en color rojo.

Ejercitación 3:

Analizar el siguiente código señalando declaraciones y aplicaciones de reglas, y su efecto.

```
p.quitar {
color: red;
}
*.desarrollo {
font-size: 8px;
}
.importante {
font-size: 20px;
}
<p class="desarrollo">
En este primer párrafo trataremos lo siguiente:
<br />xxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
</p>
<p class="quitar">
Este párrafo debe ser quitado de la obra.....
```

```

<br />xxxxxxxxxxxxxxxxxxxxxxxxxxxx
</p>
<p>
En este otro párrafo trataremos otro tema:<br />
xxxxxxxxxxxxxxxxxxxxxxxxxxxx
</p>
<p class="importante">
Y este es el párrafo más importante de la obra...
<br />xxxxxxxxxxxxxxxxxxxxxxxxxxxx
</p>
<h1 class="quitar">Este encabezado también debe ser quitado de la obra</h1>
<p class="quitar importante">Se pueden aplicar varias clases a la vez</p>

```

Análisis de las reglas CSS

p.quitar { ... }

- **Selector:** Selector de tipo (p) combinado con un selector de clase (.quitar).
- **Declaración:** Establece el color del texto en rojo.
- **Efecto:** Sólo afectará a los elementos que sean estrictamente párrafos (<p>) y que además posean el atributo class="quitar".

*.desarrollo { ... }

- **Selector:** Selector universal (*) combinado con un selector de clase (.desarrollo). El asterisco puede omitirse, siendo equivalente a escribir simplemente .desarrollo.
- **Declaración:** Establece el tamaño de la fuente en 8 píxeles.
- **Efecto:** Afectará a cualquier elemento HTML que contenga el atributo class="desarrollo", sin importar su tipo de etiqueta.

.importante { ... }

- **Selector:** Selector de clase puro.
- **Declaración:** Establece el tamaño de la fuente en 20 píxeles.
- **Efecto:** Aplicará este tamaño de letra a cualquier elemento HTML que contenga el atributo class="importante".

Análisis HTML

- **<p class="desarrollo">En este primer párrafo...</p>:** Coincide con la regla *.desarrollo. El texto se visualizará con un tamaño de fuente de 8px.
- **<p class="quitar">Este párrafo debe ser quitado...</p>:** Coincide con la regla p.quitar. El texto se visualizará en color rojo.
- **<p>En este otro párrafo trataremos otro tema...</p>:** No posee ningún atributo de clase, por lo que no coincide con ninguna regla declarada. Se visualizará con el estilo predeterminado del navegador.
- **<p class="importante">Y este es el párrafo más importante...</p>:** Coincide con la regla .importante. El texto se visualizará con un tamaño de fuente de 20px.

- **<h1 class="quitar">Este encabezado también debe ser quitado...</h1>**: Ningún estilo se aplica. Aunque el elemento posee class="quitar", la regla p.quitar exige que el elemento sea estrictamente un párrafo <p>. Este caso ilustra la diferencia entre un selector de tipo+clase y un selector de clase puro: si la regla hubiera sido simplemente .quitar, el encabezado sí habría recibido el color rojo. Al ser p.quitar, queda excluido y conserva el estilo predeterminado del navegador para <h1>.
- **<p class="quitar importante">Se pueden aplicar varias clases a la vez</p>**: El elemento coincide simultáneamente con dos reglas: p.quitar y .importante. Al no existir conflicto entre las propiedades declaradas (una modifica el color y la otra el tamaño de fuente), ambas se combinan. El texto se visualizará en color rojo y con un tamaño de fuente de 20px.

Ejercitación 4:

Dadas las siguientes declaraciones:

```
* {color:green; }
a:link {color:gray }
a:visited{color:blue }
a:hover {color:fuchsia }
a:active {color:red }
p {font-family: arial,Helvetica;font-size: 10px;color:black; }
.contenido{font-size: 14px;font-weight: bold; }
```

Analizar los siguientes códigos y comparar sus efectos. Explicar:

Código 1:

```
<body>
<p class="contenido" style="font-weight: normal;">
Este es un texto.....</p>
<table>
<tr>
<td>Y esta es una tabla.....</td>
</tr>
<tr>
<td><a href="http://www.google.com.ar">con un enlace</a></td>
</tr>
</table>
</body>
```

Código 2:

```
<body class="contenido">
<p> Este es un texto.....</p>
<table>
<tr>
<td>Y esta es una tabla.....</td>
</tr>
<tr>
<td><a href="http://www.google.com.ar">con un enlace</a></td>
```

```
</tr>
</table>
</body>
```

Análisis del Código 1

En este bloque, la clase `.contenido` y los estilos en línea se aplican de forma individual y directa sobre el párrafo.

El párrafo (`<p>`):

- Se ve afectado por la regla de etiqueta `p` (fuente Arial/Helvetica, color negro), que sobrescribe el color verde del selector universal `*` por tener mayor especificidad.
- Tiene la clase `.contenido`, que impone un tamaño de 14px y grosor bold. Como un selector de clase tiene mayor especificidad que uno de etiqueta, el tamaño resultante es 14px y no los 10px de la regla `p`.
- Posee además un estilo en línea (`style="font-weight: normal;"`). Al tener la máxima especificidad en la cascada, anula el bold impuesto por la clase.
- Resultado visual: Texto en Arial/Helvetica, color negro, tamaño 14px y grosor normal.

La tabla (`<table>` y `<td>`):

- No tienen ninguna clase ni estilo propio. Solo les aplica el selector universal `*`.
- Resultado visual: Texto de color verde, con tamaño y grosor predeterminados por el navegador.

El enlace (`<a>`):

- Su color está dictado exclusivamente por las pseudo-classes declaradas (`:link` gris, `:hover` fucsia, etc.), que anulan el verde del selector universal por especificidad.
- Su tamaño de fuente y grosor conservan los valores predeterminados del navegador, ya que ningún ancestro en este código le aplica reglas directas de `font-size` o `font-weight`.

2. Análisis del Código 2

En este bloque la estrategia cambia: la clase `.contenido` se aplica al elemento contenedor principal, el `<body>`.

El `<body>`:

- Al tener la clase `.contenido`, establece un tamaño de 14px y grosor bold. Por el principio de herencia, todos los elementos dentro del `<body>` intentarán heredar estas características a menos que tengan una regla propia que las contradiga.

El párrafo (`<p>`):

- Hereda el grosor bold del `<body>`.

- Tiene una regla propia de etiqueta (p) que impone un tamaño de 10px. Una regla directa, incluso de baja especificidad, siempre le “gana” a un valor heredado. Por lo tanto, su tamaño se reduce a 10px.
- Toma su color negro y tipografía de la misma regla p.
- Resultado visual: Texto en Arial/Helvetica, color negro, tamaño 10px y grosor bold.

La tabla (<table> y <td>):

- El color sigue siendo verde por el selector universal *, igual que en el código anterior. La diferencia está en el tamaño y el grosor, que ahora se heredan del <body>.
- Resultado visual: Texto de color verde, tamaño 14px y grosor bold.

El enlace (<a>):

- Al igual que la tabla, hereda el tamaño 14px y el grosor bold del <body>, a diferencia del código anterior donde conservaba los valores predeterminados del navegador. Su comportamiento de color sigue siendo dictado por las pseudo-clases.

Conclusión y Comparación

La diferencia principal entre ambos códigos radica en dónde se aplica la clase y cómo actúa la herencia:

En el párrafo: En el código 1 el texto es más grande (14px) y de grosor normal, mientras que en el código 2 es más chico(10px) pero en negrita. Esto se debe al choque entre estilos en línea y valores heredados frente a reglas directas.

En la tabla y los enlaces: En el código 1 conservan su tamaño y grosor predeterminados. En el código 2, todo el contenido se vuelve más grande (14px) y en negrita porque hereda los estilos aplicados directamente al <body>.

Ejercitación 5:

En cada caso, declarar una regla CSS que produzca el siguiente efecto:

1. Los textos enfatizados dentro de cualquier título deben ser rojos.

En HTML, los títulos van desde <h1> hasta <h6>, y el texto enfatizado se marca con la etiqueta . Para seleccionar un elemento descendiente de otro se usa el selector de descendientes, separando las etiquetas con un espacio. Se agrupan todas las combinaciones separándolas con comas para no escribir seis reglas distintas.

```
h1 em, h2 em, h3 em, h4 em, h5 em, h6 em {
  color: red;
}
```

2. Cualquier elemento que tenga asignado el atributo "href" y que esté dentro de cualquier párrafo que a su vez esté dentro de un bloque debe ser color negro.

El "bloque" genérico en HTML es el <div>, por lo que se interpreta el enunciado bajo ese elemento contenedor.

```
div p *[href] {  
  color: black;  
}
```

La regla se lee de izquierda a derecha usando el selector de descendientes: dentro de un div, se busca un p, y dentro de ese p, cualquier elemento (*) que posea el atributo [href], sin importar su valor. Los corchetes [] corresponden al selector de atributo, que selecciona elementos basándose en la presencia de un atributo específico independientemente de su contenido.

3. El texto de las listas no ordenadas que estén dentro del bloque identificado como “ultimo” debe ser amarillo pero si es un enlace a otra página debe ser azul.

Este caso requiere dos reglas distintas: una para pintar toda la lista de amarillo y otra más específica para los enlaces.

```
#ultimo ul {  
  color: yellow;  
}
```

```
#ultimo ul a {  
  color: blue;  
}
```

La primera regla usa el selector de ID #ultimo seguido de un descendiente ul, pintando todo el texto de la lista de amarillo. En cuanto a la segunda regla, dentro de esa misma lista (#ultimo ul), selecciona los enlaces a y los pinta de azul. Por las reglas de especificidad, la segunda regla le “gana” a la primera en los enlaces porque es más específica al apuntar directamente a la etiqueta a.

4. Los elementos identificados como “importante” dentro de cualquier bloque deben ser verdes, pero si están dentro de un título deben ser rojos.

El uso del plural “elementos identificados” genera una contradicción técnica, puesto que un atributo ID debe ser único e irrepetible en HTML. Para que el código sea válido, se interpreta como una clasificación múltiple, por lo que corresponde usar una clase (.importante) y no un ID. El bloque genérico es el <div>.

```
div .importante {  
  color: green;  
}
```

```
h1 .importante, h2 .importante, h3 .importante,  
h4 .importante, h5 .importante, h6 .importante {  
  color: red;
```

```
}
```

La primera regla selecciona cualquier elemento con clase `.importante` que sea descendiente de un `div` y lo pinta de verde. La segunda agrupa los seis niveles de título usando el selector de descendientes combinado con el selector de clase, de modo que si ese mismo elemento `.importante` está dentro de un título, la regla más específica lo pinta de rojo.

5. Todos los elementos `h1` que especifique el atributo `title`, cualquiera que sea su valor, deben ser azules.

Para seleccionar un elemento basándose únicamente en la presencia de un atributo, sin importar su valor, se usa el selector de atributo encerrando el nombre del atributo entre corchetes `[ ]` directamente junto al selector de elemento.

```
h1[title] {  
  color: blue;  
}
```

6. El color de los enlaces en las listas ordenadas debe ser azul para los enlaces aún no visitados, y violeta para los ya visitados y, además, no deben aparecer subrayados.

Las listas ordenadas se crean con la etiqueta `<ol>`. Se necesitan tres reglas: una base para eliminar el subrayado y dos con pseudo-clases para los estados del enlace.

```
ol a {  
  text-decoration: none;  
}
```

```
ol a:link {  
  color: blue;  
}
```

```
ol a:visited {  
  color: violet;  
}
```

La regla base `ol a` aplica `text-decoration: none` a todos los enlaces dentro de listas ordenadas, eliminando el subrayado predeterminado del navegador en ambos estados. Las dos reglas siguientes usan el selector de descendientes combinado con las pseudo-clases `:link` y `:visited` para asignar el color correspondiente según el estado del enlace.